

METER AUTOS

A Mumbai Ride-Fare Decision Tool — Tech Explainer

Portfolio prototype by Nibhrit Mohanty · MBA, IIM Mumbai · former PayPal SDE
Live demo: nibhrit.github.io/meter-autos-mumbai · Repo: github.com/nibhrit/meter-autos-mumbai

The Problem, and the Reframe

Mumbai's auto-rickshaw meter fare is government-fixed and published (Maharashtra RTO tariff): Rs. 26 minimum for the first 1.5km, then Rs. 17.14/km, +25% at night — exact, surge-proof, no guessing. Uber/Ola/Rapido pricing is proprietary and dynamic — it can only ever be estimated.

Market research surfaced an existing tool (MeterSahi) doing almost exactly the obvious version of this idea — and its own data showed the gap that justifies it barely exists for app-booked autos (Rs. 366 meter vs. Rs. 371–374 app autos, a Rs. 5–8 difference). But its app-fare numbers carried hedges like “excl. surge” and “always check the app before booking” — a confident verdict built on a number it admits is wrong exactly when the decision matters most.

That became the actual product: not another comparator, but a tool that calibrates its own confidence to how reliable its inputs actually are, and says “toss-up” instead of guessing when the gap is too close to call.

Architecture

Layer	Role
Fare Engine	Deterministic RTO meter formula + modeled Uber/Ola/Rapido auto & car rate cards. Every number reproducible, never an LLM guess.
Decision Logic	Turns fares into a verdict: gap-based confidence calibration, surge-proof anchoring, no-auto-zone handling, car-tier trap callout.
NL Assistant (Claude)	Parses messy questions into structured inputs and explains results in plain language. Never computes a fare — structurally cannot.
API (FastAPI)	/routes, /decide (free, instant), /ask (NL, costs an LLM call). Deployed on Render.

Key Design Decisions

Confidence is calibrated to price-gap size, not trip distance or surge state.
HIGH above a 25% gap, MEDIUM 10–25%, LOW below 10%. Letting the data decide when to sound confident — rather than hardcoding rules by distance — meant the surge-proof insight fell out for free: the same route flips from LOW to HIGH confidence as surge rises, with zero special-case code.

The LLM never computes a fare — structurally, not by instruction.
Claude has exactly two jobs: language-in (parse a query into structured inputs) and language-out (explain a result). The actual rupee arithmetic lives in a separate deterministic layer the model never touches — fare hallucination isn't prevented by careful prompting, it's architecturally impossible.

Refusing to guess beats guessing wrong, at every layer.
The decision engine says “toss-up” instead of picking a side below its confidence threshold. The NL layer declines to match a route rather than fuzzy-matching to a plausible wrong one (verified with an adversarial “Mars to Jupiter” query). Same principle, two layers.

The honest primary verdict can bury the original insight — so add a second signal, don't compromise the first.

Comparing the meter to the cheapest available app option is the honest call, but it's almost always an app-auto (small gap) — quietly burying the real finding that people default to booking a car (large gap on short trips). Resolved with a separate “car-tab trap” callout rather than diluting the primary number.

Eval Results

Suite	Result	What it measures
Decision engine regression	22 / 22	Verdict + confidence checked against an independently re-derived RTO formula, not the implementation testing itself.
NL high-confidence precision (primary)	100% (15/15)	Of every route Claude matched with confidence, how many were correct — a wrong confident match is the failure mode this project exists to prevent.
Correct refusal rate	100% (6/6)	Out-of-scope queries (wrong city, gibberish) correctly declined rather than guessed.

Honest caveat (stated in the product UI itself, not just here): 21 NL test cases validates the design — confidence-gated refusal works, varied phrasing is handled — it isn't a claim of bulletproof reliability. Harder adversarial cases (typos, ambiguous multi-location queries) are the natural next eval to add.

v1 Scope Cuts

Cut	Why
Live app-fare scraping / APIs	No public API for Uber/Ola/Rapido pricing exists; scraping is legally grey. Modeled fares instead, clearly labeled as estimates.
Live maps / geocoding	Curated preset routes keep the demo deterministic instead of dependent on a third external API.
Live surge feed	Surge is a user-selectable scenario — sufficient to demonstrate the surge-proof-meter mechanism without needing live data.

Not affiliated with Uber, Ola, Rapido, or any transport authority. App fares shown in the product are modeled estimates, not live prices. Built end-to-end (fare engine, decision logic, NL layer, API, frontend, evals, deployment) as a PM portfolio work sample.